# 🧮 CATEGORY 4: Variables, Outputs, Locals & Expressions

> **Difficulty:** Beginner → Intermediate | **Topics:** 11 | **Terraform Interview Mastery Series**

---

## Table of Contents


1. Input Variables — Types, Validation, Sensitive Flag
2. Output Values — Purpose, Sensitive Outputs, Cross-Module Use
3. Local Values — `locals` vs Variables, When to Use Each
4. ⚠ Variable Precedence — The Full Resolution Order
5. `tfvars`, `.auto.tfvars`, `-var`, `-var-file`
6. Expressions — Conditionals, `for`, Splat (`[*]`)
7. Key Built-in Functions
8. ⚠ Dynamic Blocks — When to Use, When to Avoid
9. ➕ `templatefile()` and `file()` Functions
10. ➕ `formatdate()` and Date/Time Functions
11. ➕ Tuple Type — How It Differs from List


---

# Topic 18: Input Variables — Types, Validation, Sensitive Flag

---

## 🔵 What It Is (Simple Terms)

Input variables are the **parameters** of your Terraform configuration. They let you write reusable, flexible configs by externalizing values that change between environments, teams, or deployments — instead of hardcoding them.

Think of them like function parameters in programming: you define the interface once, and callers supply the values.

---

## 🔵 Why It Exists — What Problem It Solves

Without variables, every environment needs a separate copy of your config with hardcoded values:

```
# ❌ Without variables — three separate files, massive duplication
# dev/main.tf:  instance_type = "t3.micro",  count = 1
# stg/main.tf:  instance_type = "t3.medium", count = 2
# prod/main.tf: instance_type = "t3.large",  count = 5
```

With variables, one config serves all environments:

```
# ✅ With variables — one config, values supplied externally
resource "aws_instance" "web" {
  instance_type = var.instance_type
  count         = var.instance_count
}
```

---

## 🔵 Full Variable Block Anatomy

```
variable "environment" {
  # ── Type Constraint ──────────────────────────────────────────
  type        = string              # Enforced at plan time

  # ── Description ──────────────────────────────────────────────
  description = "The deployment environment (dev/staging/prod)"

  # ── Default Value ────────────────────────────────────────────
  default     = "dev"              # Optional — makes the variable
optional
                                   # No default = required (must be
supplied)

  # ── Sensitive Flag ───────────────────────────────────────────
  sensitive   = false              # Set true to redact from CLI output

  # ── Validation Block ─────────────────────────────────────────
  validation {
    condition     = contains(["dev", "staging", "prod"], var.environment)
    error_message = "Environment must be one of: dev, staging, prod."
  }

  # ── Nullable (Terraform 1.1+) ────────────────────────────────
  nullable = false                 # false = null is not accepted as value
}
```

---

## 🔵 All Supported Type Constraints

### Primitive Types

```
variable "name"    { type = string  }    # "hello"
variable "count"   { type = number  }    # 42 or 3.14
variable "enabled" { type = bool    }    # true or false
```

### Collection Types

```
variable "availability_zones" {
  type    = list(string)
  default = ["us-east-1a", "us-east-1b", "us-east-1c"]
}

variable "tags" {
  type    = map(string)
  default = {
    Environment = "dev"
    Team        = "platform"
  }
}

variable "unique_names" {
  type    = set(string)              # unordered, no duplicates
  default = ["web", "api", "worker"]
}
```

### Structural Types

```
# object — fixed schema, named attributes, mixed types
variable "server_config" {
  type = object({
    instance_type = string
    count         = number
    enable_monitoring = bool
    tags          = map(string)
  })
  default = {
    instance_type     = "t3.medium"
    count             = 1
    enable_monitoring = true
    tags              = {}
  }
}

# list of objects — common for repeating resource configs
variable "subnets" {
  type = list(object({
    cidr = string
    az   = string
    tier = string
  }))
  default = [
    { cidr = "10.0.1.0/24", az = "us-east-1a", tier = "private" },
    { cidr = "10.0.2.0/24", az = "us-east-1b", tier = "private" },
  ]
}
```

```
# any — Terraform infers the type at runtime
variable "flexible_config" {
  type    = any
  default = {}
}
```

---

## 🔵 Variable Validation — Deep Dive

```
# Basic string validation
variable "environment" {
  type = string
  validation {
    condition     = contains(["dev", "staging", "prod"], var.environment)
    error_message = "Must be dev, staging, or prod."
  }
}

# Numeric range validation
variable "instance_count" {
  type = number
  validation {
    condition     = var.instance_count >= 1 && var.instance_count <= 10
    error_message = "Instance count must be between 1 and 10."
  }
}

# CIDR block format validation
variable "vpc_cidr" {
  type = string
  validation {
    condition     = can(cidrhost(var.vpc_cidr, 0))
    error_message = "Must be a valid CIDR block (e.g. 10.0.0.0/16)."
  }
}

# String prefix validation
variable "bucket_name" {
  type = string
  validation {
    condition     = startswith(var.bucket_name, "mycompany-")
    error_message = "Bucket names must start with 'mycompany-'."
  }
}

# Multiple validations on one variable
variable "instance_type" {
  type = string

  validation {
```

```
    condition     = length(var.instance_type) > 0
    error_message = "Instance type cannot be empty."
  }

  validation {
    condition     = can(regex("^(t3|t4g|m5|m6i|c5|c6i)\\.",
var.instance_type))
    error_message = "Only approved instance families allowed: t3, t4g, m5,
m6i, c5, c6i."
  }
}

# Cross-variable validation (Terraform 1.9+)
variable "max_size" {
  type = number
}

variable "min_size" {
  type = number
  validation {
    condition     = var.min_size <= var.max_size
    error_message = "min_size must be <= max_size."
  }
}
```

---

## 🔵 The `sensitive` Flag

```
# Marking a variable as sensitive
variable "db_password" {
  type        = string
  sensitive   = true              # Redacts value from CLI output
  description = "RDS master password"
}

variable "api_key" {
  type      = string
  sensitive = true
}
```

**What `sensitive = true` does:**

```
# In plan output — value is redacted
  + aws_db_instance.main
      + password = (sensitive value)          # ← not shown

# In apply output
Apply complete! Resources: 1 added.
```

```
# In outputs — also redacted if marked sensitive
Outputs:
  db_endpoint = "mydb.abc123.us-east-1.rds.amazonaws.com"
  db_password = <sensitive>                    # ← not shown in CLI
```

**What `sensitive = true` does NOT do:**

```
❌ Does NOT encrypt the value in state — it's still plaintext in tfstate
❌ Does NOT prevent the value from appearing in logs if you print it
❌ Does NOT protect it from `terraform output -json` (shows it)
✅ DOES redact it from normal CLI plan/apply output
✅ DOES redact it from the Terraform Cloud UI
✅ DOES prevent it from appearing in error messages
```

> ⚠ **State file still contains sensitive values in plaintext.** Encrypt your state backend and restrict access.

---

## 🔵 Sensitive Values Propagation

```
variable "db_password" {
  type      = string
  sensitive = true
}

locals {
  # This local also becomes sensitive — Terraform tracks sensitivity
  connection_string =
"postgresql://admin:${var.db_password}@${aws_db_instance.main.endpoint}/my
db"
}

output "connection_string" {
  value     = local.connection_string
  sensitive = true   # Must mark output sensitive if it contains sensitive
value
                     # Terraform 0.15+ errors if you forget this
}
```

---

## 🔵 `nullable` — Terraform 1.1+

```
variable "optional_config" {
  type     = string
  default  = "default-value"
  nullable = false           # Callers CANNOT pass null — default is used
instead
```

```
}

variable "truly_optional" {
  type     = string
  default  = null            # Default is null
  nullable = true            # Callers can explicitly pass null (default)
}
```

---

## 🔵 Short Interview Answer

> "Input variables are the parameters of a Terraform module — they externalize configurable values so the same config can serve multiple environments. Variables have a type constraint (string, number, bool, list, map, object, set, any), an optional default (no default makes it required), a description, a sensitive flag to redact values from CLI output, and a validation block for custom rules. The `sensitive` flag is important to understand — it redacts from CLI output but the value is still stored in plaintext in the state file, so state encryption and access control are required for real secret protection."

---

## 🔵 Real World Production Example

```
# variables.tf for a production RDS module

variable "identifier" {
  type        = string
  description = "The RDS instance identifier"
  validation {
    condition     = can(regex("^[a-z][a-z0-9-]{0,62}[a-z0-9]$",
var.identifier))
    error_message = "Identifier must be lowercase alphanumeric with
hyphens, 2-64 chars."
  }
}

variable "instance_class" {
  type        = string
  description = "RDS instance class"
  default     = "db.t3.medium"
  validation {
    condition     = can(regex("^db\\.(t3|t4g|m5|m6g|r5|r6g)\\.",
var.instance_class))
    error_message = "Only approved RDS instance families."
  }
}

variable "engine_version" {
  type        = string
  description = "PostgreSQL engine version"
  default     = "15.4"
```

```
}

variable "allocated_storage" {
  type    = number
  default = 100
  validation {
    condition     = var.allocated_storage >= 20 && var.allocated_storage
<= 65536
    error_message = "Storage must be between 20 and 65536 GB."
  }
}

variable "master_password" {
  type        = string
  sensitive   = true
  description = "Master DB password — provide via TF_VAR_master_password
or Vault"
}

variable "backup_retention_period" {
  type    = number
  default = 7
  validation {
    condition     = var.backup_retention_period >= 1 &&
var.backup_retention_period <= 35
    error_message = "Backup retention must be 1–35 days."
  }
}
```

---

## 🔵 Common Interview Questions

**Q: What is the difference between a variable with no default and one with `default = null`?**

> "A variable with no `default` is required — Terraform errors at plan time if no value is provided. A variable with `default = null` is optional — if not provided, the value is `null`, which tells the provider to use its own default for that attribute. The distinction matters: `default = null` is used when you want to pass through to the provider's default behavior, while no default forces the caller to make an explicit choice."

**Q: Does marking a variable `sensitive = true` protect it in state?**

> "No — `sensitive = true` only redacts the value from CLI plan/apply output and the Terraform Cloud UI. The value is still stored in plaintext in the state file. For real secret protection you need: encrypted state backend (S3 with SSE-KMS, Terraform Cloud with encryption), strict IAM access controls on the state bucket, and ideally not putting the secret in Terraform at all — use a data source to fetch it from Vault or AWS Secrets Manager at plan time."

**Q: Can you have multiple `validation` blocks on one variable?**

> "Yes, Terraform 1.3+ supports multiple validation blocks on a single variable. Each validation has its own `condition` and `error_message`. Terraform evaluates all of them and reports each failure separately. This is better than trying to combine multiple conditions into one expression because each validation gives a specific, actionable error message."

---

## 🔵 Gotchas & Edge Cases

- ⚠ **Sensitive value in non-sensitive output** — Terraform 0.15+ errors if you use a sensitive value in an output without marking the output `sensitive = true`. This protects against accidental leaks.
- **`object` type with optional attributes (Terraform 1.3+)** — use `optional()` for object attributes that don't need to be set: `object({ name = string, count = optional(number, 1) })`. The second arg to `optional()` is the default.
- **`any` type loses validation** — when you use `type = any`, Terraform accepts anything and doesn't validate types. Use sparingly.
- **Variables can't reference other variables** — `default = var.other_variable` is invalid. Use `locals` for computed values.
- **Validation can only reference `var.<name>`** — validation conditions can't reference other variables, locals, or data sources.

---

## 🔵 Connections to Other Concepts

- → **Topic 19 (Outputs):** Outputs expose computed values; sensitive variables propagate sensitivity to outputs
- → **Topic 20 (Locals):** Locals compute derived values from variables
- → **Topic 21 (Precedence):** Understanding how variable values are resolved
- → **Category 7 (Modules):** Variables are the interface for calling modules

---

# Topic 19: Output Values — Purpose, Sensitive Outputs, Cross-Module Use

---

## 🔵 What It Is (Simple Terms)

Output values **expose data from a Terraform configuration** to the outside world. They serve two purposes: showing useful information after `apply` (like an instance's public IP) and providing data to other Terraform configurations that call your module or read your state.

---

## 🔵 Why It Exists — What Problem It Solves

Without outputs:

- You'd have to dig through state files to find resource IDs and endpoints
- Modules would be black boxes — no way to get data out

- Cross-stack communication would be impossible
- Post-apply, you'd have to query the cloud API manually to find what was created

---

## 🔵 Full Output Block Anatomy

```
output "vpc_id" {
  # ── Value (required) ──────────────────────────────────────────
  value = aws_vpc.main.id

  # ── Description (recommended) ─────────────────────────────────
  description = "The ID of the main VPC"

  # ── Sensitive flag ────────────────────────────────────────────
  sensitive = false

  # ── depends_on (rare) ─────────────────────────────────────────
  # Ensures output is only available after dependencies complete
  depends_on = [aws_vpc.main]   # usually not needed — implied by value
reference
}
```

---

## 🔵 Common Output Patterns

### Basic Outputs

```
# Single resource attribute
output "instance_id" {
  description = "The EC2 instance ID"
  value       = aws_instance.web.id
}

output "instance_public_ip" {
  description = "Public IP — may change on stop/start"
  value       = aws_instance.web.public_ip
}

output "rds_endpoint" {
  description = "RDS connection endpoint"
  value       = aws_db_instance.main.endpoint
}
```

### Collection Outputs

```
# List of values from count-based resources
output "instance_ids" {
```

```
  description = "All instance IDs"
  value       = aws_instance.web[*].id          # splat expression
}

# Map of values from for_each-based resources
output "instance_map" {
  description = "Map of instance name -> ID"
  value       = {
    for k, v in aws_instance.web : k => v.id
  }
}

# List of subnet IDs
output "private_subnet_ids" {
  description = "Private subnet IDs for use by app stacks"
  value       = aws_subnet.private[*].id
}
```

### Structured Outputs

```
# Composite output — bundle related data
output "database_config" {
  description = "Database connection configuration"
  value = {
    endpoint = aws_db_instance.main.endpoint
    port     = aws_db_instance.main.port
    name     = aws_db_instance.main.db_name
    username = aws_db_instance.main.username
  }
  sensitive = true   # mark entire object sensitive if any value is
sensitive
}

output "load_balancer" {
  description = "Load balancer details"
  value = {
    dns_name = aws_lb.main.dns_name
    zone_id  = aws_lb.main.zone_id
    arn      = aws_lb.main.arn
  }
}
```

---

## 🔵 Sensitive Outputs

```
output "db_password" {
  value     = random_password.db.result
  sensitive = true
  # This output:
```

```
  # ✅ Shows as <sensitive> in terraform apply output
  # ✅ Redacted in Terraform Cloud UI
  # ❌ Still accessible via: terraform output -raw db_password
  # ❌ Still in state file in plaintext
}
```

```
# Accessing sensitive outputs explicitly
terraform output db_password             # Error: output is sensitive
terraform output -raw db_password        # Returns the value (overrides
sensitive flag)
terraform output -json | jq '.db_password.value'  # Returns value via JSON

# In CI/CD — sensitive outputs show as (sensitive value) in logs
# unless explicitly printed
```

---

## 🔵 Outputs in Root Module vs Child Module

### Root Module Outputs

```
# After terraform apply, outputs are printed:
Apply complete! Resources: 3 added, 0 changed, 0 destroyed.

Outputs:
  instance_public_ip = "54.12.34.56"
  rds_endpoint       = "mydb.abc123.us-east-1.rds.amazonaws.com:5432"
  vpc_id             = "vpc-0a1b2c3d4e5f67890"

# Query outputs after apply:
terraform output                    # all outputs
terraform output vpc_id             # specific output
terraform output -raw vpc_id        # raw value (no quotes)
terraform output -json              # all outputs as JSON
```

### Child Module Outputs (Cross-Module Communication)

```
# modules/vpc/outputs.tf
output "vpc_id" {
  value = aws_vpc.main.id
}

output "private_subnet_ids" {
  value = aws_subnet.private[*].id
}

# Root module using the child module
module "vpc" {
```

```
  source = "./modules/vpc"
  cidr   = "10.0.0.0/16"
}

# Accessing module outputs:
resource "aws_instance" "web" {
  subnet_id = module.vpc.private_subnet_ids[0]    # module.<name>.<output>
  vpc_security_group_ids = [module.security_groups.web_sg_id]
}

output "vpc_id" {
  value = module.vpc.vpc_id     # re-export module output to root
}
```

---

## 🔵 Output Dependency Behavior

```
# Outputs are evaluated AFTER all resources are created
# They can reference computed attributes

output "nat_gateway_ips" {
  # These IPs are only known after apply
  value = aws_eip.nat[*].public_ip
}

# Outputs with depends_on (rare but valid)
output "deployment_complete" {
  value      = "Application deployed successfully"
  depends_on = [aws_ecs_service.app, aws_lb_listener.https]
  # Ensures output only shows after ECS service and LB are ready
}
```

---

## 🔵 Cross-Stack Communication Patterns

Pattern 1: `terraform_remote_state` (Tight Coupling)

```
# networking/outputs.tf
output "vpc_id" { value = aws_vpc.main.id }
output "private_subnet_ids" { value = aws_subnet.private[*].id }

# application/main.tf
data "terraform_remote_state" "networking" {
  backend = "s3"
  config  = {
    bucket = "mycompany-tf-state"
    key    = "networking/terraform.tfstate"
    region = "us-east-1"
  }
```

```
}

resource "aws_instance" "app" {
  subnet_id =
data.terraform_remote_state.networking.outputs.private_subnet_ids[0]
}
```

Pattern 2: SSM Parameter Store (Loose Coupling — Recommended)

```
# networking stack — write to SSM
resource "aws_ssm_parameter" "vpc_id" {
  name  = "/networking/prod/vpc_id"
  type  = "String"
  value = aws_vpc.main.id
}

# application stack — read from SSM (no state dependency)
data "aws_ssm_parameter" "vpc_id" {
  name = "/networking/prod/vpc_id"
}

resource "aws_instance" "app" {
  subnet_id = data.aws_ssm_parameter.subnet_id.value
}
```

---

## 🔵 Short Interview Answer

> "Outputs expose values from a Terraform config — either to display after apply or for use by other modules and stacks. In root modules they print to terminal and are accessible via `terraform output`. In child modules, other configs reference them as `module.<name>.<output>`. Sensitive outputs are redacted from CLI display with `sensitive = true` but are still in state, still accessible with `terraform output -raw`, and still need state encryption for real protection. For cross-stack sharing, I prefer SSM Parameter Store over `terraform_remote_state` because it decouples the stacks — the app stack doesn't need read access to the networking state bucket."

---

## 🔵 Common Interview Questions

**Q: How do you share data between two separate Terraform state files?**

> "Two main patterns: `terraform_remote_state` reads another state file's outputs directly — tight coupling but simple. SSM Parameter Store (or Consul, or similar) — the producing stack writes values to SSM, the consuming stack reads via a data source. SSM is preferred in production because it decouples the stacks, doesn't require cross-team state access, and parameters can be versioned. The producing stack can change its internal implementation without breaking consumers as long as it still writes the same SSM parameters."

**Q: What happens to module outputs if the module has no resources?**

> "If a module has no resources (or all its resources are destroyed), outputs that reference resource attributes will be `null`. Outputs with static values will still work. If a root module references a child module output that resolves to `null`, dependent resources may fail with 'expected string, got null' type errors. Design your module outputs defensively — use `try()` or conditional expressions to handle cases where resources might not exist."

---

## 🔵 Gotchas & Edge Cases

- ⚠ **Outputs are recalculated on every apply** — if an output references a resource that changed, the output updates. If you've piped outputs to other systems, be prepared for updates.
- ⚠ **Removing an output from state used by `remote_state`** — if another stack reads `output.vpc_id` and you remove that output, the consuming stack breaks. Treat outputs as a public API.
- **`terraform output` requires state** — you can't use it in a fresh directory without running apply first.
- **Sensitive output in `remote_state`** — when reading another stack's outputs via `terraform_remote_state`, sensitive outputs from that stack are NOT automatically marked sensitive in the reading stack. Manually mark them.

---

## 🔵 Connections to Other Concepts

- → **Topic 18 (Variables):** Sensitive variable values propagate sensitivity to outputs
- → **Topic 20 (Locals):** Locals transform data before it's exposed via outputs
- → **Category 7 (Modules):** Outputs are the public API of a module
- → **Category 6 (State):** Outputs are stored in state; `remote_state` reads them

---

# Topic 20: Local Values — `locals` vs Variables, When to Use Each

---

## 🔵 What It Is (Simple Terms)

Local values are **computed, reusable expressions within a module**. They're like local variables in a function — calculated once, referenced many times, and not exposed outside the module.

```
locals {
  common_tags = {
    Environment = var.environment
    ManagedBy   = "terraform"
    Repository  = "github.com/myorg/infra"
    Owner       = var.team
  }
```

```
  name_prefix = "${var.project}-${var.environment}"
  is_prod     = var.environment == "prod"
}
```

---

## 🔵 Why It Exists — What Problem It Solves

Without locals:

```
# ❌ Without locals — repetition everywhere
resource "aws_instance" "web" {
  tags = {
    Name        = "${var.project}-${var.environment}-web"
    Environment = var.environment
    ManagedBy   = "terraform"
    Repository  = "github.com/myorg/infra"
  }
}

resource "aws_s3_bucket" "data" {
  tags = {
    Name        = "${var.project}-${var.environment}-data"
    Environment = var.environment
    ManagedBy   = "terraform"
    Repository  = "github.com/myorg/infra"
  }
}
# Repeat for every resource... maintain in 20 places
```

With locals:

```
# ✅ With locals — defined once, used everywhere
locals {
  common_tags = {
    Environment = var.environment
    ManagedBy   = "terraform"
    Repository  = "github.com/myorg/infra"
  }
  name_prefix = "${var.project}-${var.environment}"
}

resource "aws_instance" "web" {
  tags = merge(local.common_tags, { Name = "${local.name_prefix}-web" })
}

resource "aws_s3_bucket" "data" {
  tags = merge(local.common_tags, { Name = "${local.name_prefix}-data" })
}
```

---

## 🔵 Locals Syntax

```
# Single locals block (can have multiple per module)
locals {
  # Simple computed value
  name_prefix = "${var.project}-${var.environment}"

  # Conditional
  instance_type = var.environment == "prod" ? "t3.large" : "t3.micro"

  # Map construction
  common_tags = {
    Environment = var.environment
    ManagedBy   = "terraform"
    Team        = var.team_name
  }

  # List transformation
  azs = slice(data.aws_availability_zones.available.names, 0,
var.az_count)

  # Complex expression
  private_subnets = [
    for i, az in local.azs :
    cidrsubnet(var.vpc_cidr, 4, i)
  ]

  # Derived from other locals (locals CAN reference other locals)
  full_name = "${local.name_prefix}-${var.component}"

  # Boolean flag
  is_production = var.environment == "prod"
  has_multi_az  = local.is_production
}
```

---

## 🔵 Variables vs Locals vs Outputs — The Full Comparison

```
┌──────────────────────────────────────────────────────────────────┐
│           Variables vs Locals vs Outputs                         │
│                                                                  │
│  VARIABLES (input)                                               │
│  ✅ Accept values from outside the module                        │
│  ✅ Have type constraints and validation                         │
│  ✅ Can have defaults                                            │
│  ❌ Cannot reference other variables or resources                │
│  ❌ Cannot compute derived values                                │
```

```
│  Use when: caller needs to configure the module                    │
│                                                                     │
│  LOCALS (internal)                                                  │
│  ✅ Compute derived values from vars, resources, data sources       │
│  ✅ Reduce repetition within a module                               │
│  ✅ Can reference other locals, variables, resources                │
│  ❌ Not visible outside the module                                  │
│  ❌ Cannot be set from outside                                      │
│  Use when: you have a reusable computed expression                  │
│                                                                     │
│  OUTPUTS (export)                                                   │
│  ✅ Expose values to callers of the module                          │
│  ✅ Display values after apply                                      │
│  ✅ Enable cross-stack communication                                │
│  ❌ Cannot be used as input                                         │
│  Use when: callers need data from the module                        │
└─────────────────────────────────────────────────────────────────────┘
```

---

## 🔵 Decision Framework: Variable vs Local

```
Q: Does the value come from outside the module?
  YES → Variable

Q: Is it a computed expression derived from variables/resources?
  YES → Local

Q: Does it need to be configured differently per caller?
  YES → Variable

Q: Is it repeated in multiple places within the module?
  YES → Local

Q: Is it a business rule or convention internal to the module?
  YES → Local (not a variable — callers shouldn't override it)

Q: Does the caller need to see this value?
  YES → Output
```

---

## 🔵 Advanced Locals Patterns

```
locals {
  # Environment-specific config maps
  env_config = {
    dev = {
      instance_type = "t3.micro"
      min_size      = 1
      max_size      = 2
```

```
      multi_az      = false
    }
    staging = {
      instance_type = "t3.medium"
      min_size      = 2
      max_size      = 4
      multi_az      = true
    }
    prod = {
      instance_type = "t3.large"
      min_size      = 3
      max_size      = 10
      multi_az      = true
    }
  }

  # Look up config for current environment
  config = local.env_config[var.environment]

  # Use: local.config.instance_type, local.config.min_size, etc.
}

# Subnet CIDR calculation
locals {
  vpc_cidr = "10.${var.vpc_number}.0.0/16"

  private_cidrs = [
    for i in range(var.az_count) :
    cidrsubnet(local.vpc_cidr, 8, i)
  ]

  public_cidrs = [
    for i in range(var.az_count) :
    cidrsubnet(local.vpc_cidr, 8, i + 100)
  ]
}
```

---

## 🔵 Short Interview Answer

> "Locals are named, computed expressions within a module — they reduce repetition and encode derived logic. Unlike variables, they're not settable from outside and can reference other locals, variables, and resource attributes. The classic use case is common tags — define once in locals, merge into every resource's tags. The decision rule: use a variable when the caller needs to configure the value, use a local when it's a derived expression you want to reuse internally, use an output when you want to expose a value to callers."

---

## 🔵 Gotchas & Edge Cases

- ⚠ **No circular references in locals** — `local.a = local.b` and `local.b = local.a` causes an error. Terraform detects cycles in the local value graph.
- **Multiple `locals` blocks are fine** — you can have several `locals {}` blocks in a module. They're all merged. Useful for organizing locals by concern.
- **Locals are not lazy** — all locals are evaluated during plan regardless of whether they're referenced. If a local expression errors (e.g. index out of bounds), the plan fails.
- **Locals are module-scoped** — you can't access `local.something` from a parent or child module. Each module has its own local namespace.

---

## 🔵 Connections to Other Concepts

- → **Topic 18 (Variables):** Locals compute derived values from variables
- → **Topic 23 (Expressions):** Locals use all expression features — `for`, conditionals, functions
- → **Topic 24 (Functions):** `merge()`, `concat()`, `cidrsubnet()` are commonly used in locals

---

# Topic 21: ⚠ Variable Precedence — The Full Resolution Order

---

## 🔵 What It Is (Simple Terms)

When the same variable can be set in multiple ways — environment variable, `.tfvars` file, `-var` flag, default value — Terraform has a strict precedence order determining which value wins. Getting this wrong leads to mysterious behavior where you think you're setting one value but a higher-priority source overrides it.

> ⚠ This is a frequent interview question and a common source of real-world bugs. Most candidates know some of the order but miss the full chain.

---

## 🔵 The Full Precedence Order (Lowest to Highest)

```
┌──────────────────────────────────────────────────────────────────────┐
│          Variable Precedence — Lowest to Highest Priority            │
│                                                                      │
│  1. Default value in variable block          ← LOWEST PRIORITY       │
│     variable "env" { default = "dev" }                               │
│                                                                      │
│  2. terraform.tfvars (in working directory)                          │
│     env = "staging"                                                  │
│                                                                      │
│  3. terraform.tfvars.json (in working directory)                     │
│     { "env": "staging" }                                             │
│                                                                      │
│  4. *.auto.tfvars files (alphabetical order)                         │
```

```
│     01-base.auto.tfvars, 02-env.auto.tfvars                          │
│                                                                       │
│  5. *.auto.tfvars.json files (alphabetical order)                    │
│                                                                       │
│  6. -var-file flag (command line)                                    │
│     terraform apply -var-file="prod.tfvars"                          │
│                                                                       │
│  7. -var flag (command line)                                         │
│     terraform apply -var="env=prod"                                  │
│                                                                       │
│  8. TF_VAR_<name> environment variables       ← HIGHEST PRIORITY    │
│     export TF_VAR_env=prod                                           │
└──────────────────────────────────────────────────────────────────────┘
```

> ⚠ **Common misconception:** Many candidates think `TF_VAR_` has lower priority than `-var`. It's the opposite — environment variables have the HIGHEST priority in most situations.

---

## 🔵 Each Method Explained

### 1. Default in Variable Block

```
variable "environment" {
  type    = string
  default = "dev"    # Used only if nothing else provides a value
}
```

### 2 & 3. `terraform.tfvars` / `terraform.tfvars.json`

```
# terraform.tfvars — automatically loaded, no flag needed
environment     = "staging"
instance_count  = 2
instance_type   = "t3.medium"
```

```
// terraform.tfvars.json — JSON alternative, also auto-loaded
{
  "environment": "staging",
  "instance_count": 2
}
```

### 4 & 5. `*.auto.tfvars` Files

```
# production.auto.tfvars — automatically loaded (no flag needed)
# Loaded in alphabetical order
```

```
environment = "prod"
min_size    = 3
```

> ⚠ Auto-loaded files can cause surprise overrides. If someone adds `prod.auto.tfvars` in the working directory, it silently overrides `terraform.tfvars`.

6. `-var-file` Flag

```
# Explicitly loaded, ONLY when specified
terraform apply -var-file="environments/prod.tfvars"
terraform apply -var-file="secrets.tfvars" -var-file="prod.tfvars"
# Multiple -var-file flags: last one wins for same variable
```

7. `-var` Flag

```
# Inline override — highest priority among file-based methods
terraform apply -var="environment=prod"
terraform apply -var="instance_count=5" -var="instance_type=t3.large"
# Multiple -var for same variable: LAST one wins
terraform apply -var="count=3" -var="count=5"   # count = 5
```

8. `TF_VAR_` Environment Variables

```
# Environment variables — highest overall priority
export TF_VAR_environment="prod"
export TF_VAR_instance_count="5"
export TF_VAR_db_password="super-secret-password"    # good for secrets
```

> **Note on types:** `TF_VAR_` for complex types (list, map) uses HCL syntax:
>
> ```
> export TF_VAR_availability_zones='["us-east-1a","us-east-1b"]'
> export TF_VAR_tags='{"env":"prod","team":"platform"}'
> ```

---

## 🔵 Precedence in Practice — A Worked Example

```
# variable.tf
variable "instance_type" {
  type    = string
  default = "t3.micro"          # Priority 1: default
}
```

```
# terraform.tfvars
instance_type = "t3.small"      # Priority 2: overrides default
```

```
# prod.auto.tfvars
instance_type = "t3.medium"     # Priority 4: overrides tfvars
```

```
# Command line
export TF_VAR_instance_type="t3.large"    # Priority 8: wins!

terraform apply -var="instance_type=t3.xlarge"  # Priority 7: but TF_VAR
wins over -var!

# FINAL VALUE: t3.large (TF_VAR beats -var)
```

---

## 🔵 `-var-file` vs `-var` Priority

```
# -var-file loads file (priority 6)
# -var is inline (priority 7)
# -var WINS over -var-file for same variable

terraform apply \
  -var-file="base.tfvars" \      # sets instance_type = "t3.medium"
  -var="instance_type=t3.large"  # wins — final value is t3.large
```

---

## 🔵 CI/CD Usage Pattern

```
# Recommended pattern: base tfvars + environment override
terraform apply \
  -var-file="environments/base.tfvars" \     # shared defaults
  -var-file="environments/${ENV}.tfvars" \   # env-specific overrides
  -var="image_tag=${DOCKER_IMAGE_TAG}"        # dynamic at deploy time

# Secrets via environment variables (never in files)
export TF_VAR_db_password="${DB_PASSWORD}"   # injected by CI/CD
export TF_VAR_api_key="${API_KEY}"
```

---

## 🔵 Short Interview Answer

> "Terraform resolves variable values in this priority order, lowest to highest: default in the variable block, `terraform.tfvars`, `terraform.tfvars.json`, `*.auto.tfvars` files alphabetically, `*.auto.tfvars.json` files alphabetically, `-var-file` flags, `-var` inline flags, and `TF_VAR_` environment variables which have the highest priority. A common misconception is that `-var` overrides environment variables — it doesn't. `TF_VAR_` wins over everything. In CI/CD, I use this: base tfvars for defaults, env-specific tfvars for environment overrides, and `TF_VAR_` for secrets since they're never written to files."

---

## 🔵 Common Interview Questions

**Q: What's the highest-priority way to set a Terraform variable?**

> "`TF_VAR_<name>` environment variables have the highest priority — they override everything including `-var` flags. This is important for CI/CD secret injection: set secrets as environment variables (`TF_VAR_db_password`), and they'll override any accidental default or file-based values. It also means if a CI system injects `TF_VAR_` variables, they can't be accidentally overridden by tfvars files."

**Q: What is `.auto.tfvars` and when would you use it?**

> "Files matching `*.auto.tfvars` are automatically loaded by Terraform without needing a `-var-file` flag — like `terraform.tfvars` but with a custom name. They're loaded in alphabetical order. Use case: in a repo with multiple environments, you might have `dev.auto.tfvars` in the dev workspace directory and `prod.auto.tfvars` in prod. Gotcha: since they're auto-loaded based on filename, adding one to a directory can silently override values — always check for existing `.auto.tfvars` files when troubleshooting unexpected variable values."

---

## 🔵 Gotchas & Edge Cases

- ⚠ **`TF_VAR_` beats `-var`** — not intuitive. If CI sets `TF_VAR_env=prod` and you try `-var="env=dev"`, the TF_VAR wins.
- ⚠ **`.auto.tfvars` loaded silently** — files in the working directory are automatically picked up. Be careful with `.auto.tfvars` files in shared or templated repos.
- **Multiple `-var-file` flags** — for the same variable, last `-var-file` wins (later on the command line overrides earlier).
- **No variable = required at plan** — if no default and no value source provides a value, Terraform prompts interactively in a terminal, or errors in non-interactive CI.
- **`-var` doesn't work for complex types in all versions** — complex type variables (list, map) are best set via tfvars files, not inline `-var` flags, due to shell escaping complexity.

---

## 🔵 Connections to Other Concepts

- → **Topic 22 (`tfvars`):** Deep dive into the file-based methods
- → **Category 8 (Security):** `TF_VAR_` is the secure way to inject secrets
- → **Category 9 (CI/CD):** Precedence understanding is essential for CI/CD variable management

---

# Topic 22: `tfvars`, `.auto.tfvars`, `-var`, `-var-file`

---

## 🔵 What It Is (Simple Terms)

These are the **file and flag based mechanisms** for supplying values to Terraform variables. Each has a different use case, different loading behavior, and different priority.

---

## 🔵 `terraform.tfvars`

```
# terraform.tfvars — the default variable values file
# Automatically loaded when present in working directory
# No flag needed

# Simple values
environment    = "production"
aws_region     = "us-east-1"
instance_count = 3
enable_backups = true

# List values
availability_zones = ["us-east-1a", "us-east-1b", "us-east-1c"]

# Map values
tags = {
  Team    = "platform"
  Project = "infra"
}

# Object values
server_config = {
  instance_type     = "t3.large"
  count             = 5
  enable_monitoring = true
}
```

**When to use:** Shared default values that apply to everyone using this configuration. Should be committed to Git (unless it contains secrets).

---

## 🔵 `*.auto.tfvars` Pattern

```
# base.auto.tfvars — auto-loaded, shared defaults
aws_region  = "us-east-1"
owner_email = "platform@mycompany.com"
```

```
# prod.auto.tfvars — auto-loaded, env-specific
environment    = "prod"
instance_type  = "t3.large"
instance_count = 10
```

**Naming pattern for multi-env repos:**

```
environments/
├── dev/
│   ├── main.tf -> ../../main.tf (symlink)
│   └── terraform.tfvars
├── staging/
│   ├── main.tf -> ../../main.tf
│   └── terraform.tfvars
└── prod/
    ├── main.tf -> ../../main.tf
    └── terraform.tfvars
```

---

## 🔵 `-var-file` Flag Pattern

```
# environments/prod.tfvars — loaded only with -var-file flag
environment    = "prod"
instance_type  = "t3.large"
instance_count = 10
domain_name    = "app.mycompany.com"
```

```
# Usage patterns:
terraform plan -var-file="environments/prod.tfvars"

# Layered approach — base + environment + region specific
terraform apply \
  -var-file="base.tfvars" \
  -var-file="environments/prod.tfvars" \
  -var-file="regions/us-east-1.tfvars"

# Secrets file — NOT committed to Git
terraform apply \
  -var-file="prod.tfvars" \
  -var-file="secrets.tfvars"    # .gitignore this file
```

---

## 🔵 `-var` Flag Pattern

```
# Individual variable override
terraform apply -var="environment=prod"
terraform apply -var="instance_count=5"

# Multiple variables
terraform apply \
  -var="environment=prod" \
  -var="instance_count=5" \
  -var="docker_image_tag=v1.2.3"

# Dynamic values (common in CI/CD)
terraform apply \
  -var-file="prod.tfvars" \
  -var="image_tag=${GIT_SHA}"     # inject build artifact tag

# Complex type via -var (requires careful quoting)
terraform apply -var='tags={"env":"prod","team":"platform"}'
```

---

## 🔵 `tfvars.json` Format

```
// terraform.tfvars.json — JSON alternative to HCL tfvars
{
  "environment": "production",
  "instance_count": 3,
  "availability_zones": ["us-east-1a", "us-east-1b"],
  "tags": {
    "Team": "platform",
    "Project": "infra"
  },
  "server_config": {
    "instance_type": "t3.large",
    "count": 5,
    "enable_monitoring": true
  }
}
```

**When to use JSON format:** When your CI/CD system generates variable files programmatically (easier to generate valid JSON than HCL), or when integrating with tools that produce JSON output.

---

## 🔵 What NOT to Put in `tfvars` Files

```
# ❌ NEVER put these in committed tfvars files:
db_password       = "super-secret"         # Use TF_VAR_ env var
api_key           = "sk-abc123xyz"         # Use TF_VAR_ env var
aws_secret_key    = "wJalrXUtnFE..."      # Use TF_VAR_ or instance role
private_key_pem   = "-----BEGIN RSA..."   # Use TF_VAR_ or Vault
```

```
# ✅ Safe to commit:
environment    = "prod"
aws_region     = "us-east-1"
instance_count = 3
vpc_cidr       = "10.0.0.0/16"
```

`.gitignore` **pattern:**

```
# .gitignore
*.tfvars          # ignore ALL tfvars (too broad — use carefully)
secrets.tfvars    # ignore only secret files (better)
*.tfvars.json     # ignore JSON tfvars
!terraform.tfvars # but don't ignore the main one (if it has no secrets)
```

---

## 🔵 Short Interview Answer

> "`terraform.tfvars` is automatically loaded and holds shared defaults. `*.auto.tfvars` files are also auto-loaded in alphabetical order — useful for environment-specific auto-overrides. `-var-file` explicitly loads a named file and is used for environment configuration in CI/CD. `-var` sets individual values inline, useful for dynamic values like image tags. The key security rule is never putting secrets in tfvars files — use `TF_VAR_` environment variables for secrets so they're injected by the CI/CD system without ever being written to disk."

---

## 🔵 Gotchas & Edge Cases

- ⚠ **`terraform.tfvars` is silently loaded** — if it exists in the directory, it's always used. Easy to forget it's there when troubleshooting.
- ⚠ **`secrets.tfvars` in Git history** — even if you delete and `.gitignore` a secrets file, the secret is in git history. Use `git filter-branch` or `BFG Repo-Cleaner` to purge.
- **`-var-file` paths are relative to current directory** — not relative to the `.tf` files. Always run Terraform from the module directory.
- **HCL in tfvars ≠ full HCL** — tfvars files only support variable assignment syntax. You can't use `resource`, `locals`, or functions in a tfvars file.

---

# Topic 23: Expressions — Conditionals, `for`, Splat (`[*]`)

---

## 🔵 What It Is (Simple Terms)

Expressions are the building blocks for computing values in Terraform — they go beyond simple literal values to enable logic, transformation, and dynamic config generation.

---

## 🔵 Conditional Expressions

```
# Syntax: condition ? true_value : false_value

# Simple boolean condition
instance_type = var.environment == "prod" ? "t3.large" : "t3.micro"

# Null coalescing pattern
subnet_id = var.custom_subnet_id != null ? var.custom_subnet_id :
aws_subnet.default.id

# Count-based conditional (create resource or not)
resource "aws_eip" "nat" {
  count  = var.enable_nat_gateway ? 1 : 0   # create EIP only if NAT
needed
  domain = "vpc"
}

# Nested conditional (keep shallow — deep nesting is unreadable)
instance_type = var.env == "prod" ? "t3.large" : (var.env == "staging" ?
"t3.medium" : "t3.micro")
# ↑ Better to use lookup() or local map for this pattern

# Better pattern for multi-value conditionals:
locals {
  instance_types = {
    prod    = "t3.large"
    staging = "t3.medium"
    dev     = "t3.micro"
  }
  instance_type = local.instance_types[var.environment]
  # Or with a fallback:
  instance_type_safe = lookup(local.instance_types, var.environment,
"t3.micro")
}
```

---

## 🔵 `for` Expressions

```
# List for expression
# [for <item> in <list> : <output_expression>]
locals {
  upper_names = [for name in var.names : upper(name)]
  # var.names = ["web", "api", "worker"]
  # result:   ["WEB", "API", "WORKER"]
```

```
  # With condition (filter)
  enabled_services = [for s in var.services : s.name if s.enabled]

  # Transform list of objects
  instance_names = [for inst in var.instances :
"${var.prefix}-${inst.name}"]
}

# Map for expression
# {for <key>, <value> in <map> : <new_key> => <new_value>}
locals {
  # Transform map values
  upper_tags = {for k, v in var.tags : k => upper(v)}

  # Invert a map (keys become values)
  tag_keys = {for k, v in var.tags : v => k}

  # Build a map from a list of objects
  instance_map = {for inst in var.instances : inst.name => inst.id}
  # result: { "web" = "i-abc", "api" = "i-def" }

  # Filter map entries
  prod_resources = {for k, v in var.resources : k => v if v.environment ==
"prod"}
}

# for expression with index
locals {
  indexed = [for i, v in var.names : "${i}: ${v}"]
  # result: ["0: web", "1: api", "2: worker"]
}
```

---

## 🔵 Splat Expressions

```
# Splat: shorthand for [for o in list : o.attribute]
# Legacy splat [*] — works on lists from count
output "instance_ids" {
  value = aws_instance.web[*].id         # collect .id from all instances
  # Equivalent to: [for inst in aws_instance.web : inst.id]
}

output "instance_ips" {
  value = aws_instance.web[*].public_ip
}

# Attribute splat [*] — works on lists and sets
locals {
  all_sg_ids = aws_security_group.web[*].id
  all_arns   = aws_iam_role.workers[*].arn
}
```

```
# Full splat — also applies method-style attribute access
output "subnet_cidrs" {
  value = aws_subnet.private[*].cidr_block
}

# ⚠ Splat doesn't work directly with for_each resources
# for_each returns a map, not a list
# Use values() + for instead:
output "for_each_ids" {
  value = [for k, v in aws_instance.web : v.id]
  # Or: values(aws_instance.web)[*].id
}
```

---

## 🔵 String Expressions

```
# Basic interpolation
name = "web-${var.environment}"

# Multi-line heredoc
user_data = <<-EOT
  #!/bin/bash
  echo "Environment: ${var.environment}"
  apt-get update
  apt-get install -y nginx
EOT

# templatefile() replaces heredoc for complex templates (see Topic 26)
```

---

## 🔵 Short Interview Answer

> "Terraform supports three main expression types beyond literals: conditionals using the ternary operator `condition ? true_val : false_val`, `for` expressions that transform lists into lists or maps, and splat expressions `[*]` as shorthand for collecting an attribute across all instances of a count-based resource. For multi-value conditionals, a local map with `lookup()` is cleaner than nested ternaries. Splat doesn't work directly on `for_each` resources since they return maps — use `values()` and a `for` expression instead."

---

## 🔵 Gotchas & Edge Cases

- ⚠ **Splat on `for_each` resources fails** — `aws_instance.web[*].id` only works for `count`-based resources. For `for_each`, use `[for k, v in aws_instance.web : v.id]` or `values(aws_instance.web)[*].id`.
- ⚠ **`for` expression result type** — `[for ...]` always produces a list (or tuple). `{for ...}` always produces a map (or object). They can't be mixed.

- **`for` expression with null values** — if any item in the list is null, accessing `.attribute` on it will error. Use `try()` or filter nulls first.
- **Empty `for` expressions** — `[for x in [] : x]` returns `[]`, not null. This is usually desired behavior.

---

# Topic 24: Key Built-in Functions

---

## 🔵 What It Is (Simple Terms)

Terraform has dozens of built-in functions for transforming and working with values. You can't define custom functions (that's what `locals` and modules are for), but the built-in library covers most needs.

---

## 🔵 The Must-Know Functions

### `lookup` — Safe Map Access

```
# lookup(map, key, default)
# Returns value for key, or default if key doesn't exist

locals {
  instance_type = lookup(var.instance_types, var.environment, "t3.micro")
  # var.instance_types = { dev = "t3.micro", prod = "t3.large" }
  # If var.environment = "staging" (not in map): returns "t3.micro"

  # Without default — errors if key missing:
  # var.instance_types[var.environment]  ← errors on missing key
  # lookup(var.instance_types, var.environment)  ← also errors
}
```

---

### `merge` — Combine Maps

```
# merge(map1, map2, ...) — later maps override earlier ones

locals {
  base_tags = { ManagedBy = "terraform", Environment = var.env }
  extra_tags = { Team = var.team, CostCenter = var.cost_center }

  all_tags = merge(local.base_tags, local.extra_tags)
  # { ManagedBy="terraform", Environment="prod", Team="platform",
CostCenter="123" }

  # Override pattern — resource-specific tags win over defaults
  resource_tags = merge(
```

```
    local.base_tags,
    { Name = "web-server", Service = "web" }
  )
}

resource "aws_instance" "web" {
  tags = merge(local.common_tags, { Name = "${local.prefix}-web" })
}
```

---

### `flatten` — Flatten Nested Lists

```
# flatten(list_of_lists) → single flat list

locals {
  # Problem: module outputs a list of lists
  all_subnet_ids = flatten([
    module.vpc_primary.private_subnet_ids,    # ["subnet-1", "subnet-2"]
    module.vpc_secondary.private_subnet_ids,  # ["subnet-3", "subnet-4"]
  ])
  # Result: ["subnet-1", "subnet-2", "subnet-3", "subnet-4"]

  # Common pattern: flatten list of objects with nested lists
  all_rules = flatten([
    for sg in var.security_groups : [
      for rule in sg.rules : {
        sg_name = sg.name
        port    = rule.port
        cidr    = rule.cidr
      }
    ]
  ])
}
```

---

### `toset` — Convert List to Set

```
# toset(list) → set (removes duplicates, loses order)
# Critical for for_each which requires a set or map

resource "aws_iam_user_group_membership" "admin" {
  for_each = toset(var.admin_users)       # convert list to set for
for_each
  # var.admin_users = ["alice", "bob", "alice"]  ← duplicate
  # toset result: {"alice", "bob"}              ← deduped

  user   = each.key
  groups = ["admins"]
}
```

```
locals {
  # Convert list to set to remove duplicates
  unique_azs = toset(var.availability_zones)

  # tolist(set) - convert back to list (order not guaranteed)
  az_list = tolist(local.unique_azs)
}
```

---

### `coalesce` and `coalescelist` — First Non-Null Value

```
# coalesce(val1, val2, ...) - returns first non-null, non-empty string

locals {
  # Use provided name or fall back to auto-generated
  instance_name = coalesce(var.instance_name,
"web-${var.environment}-${random_id.suffix.hex}")

  # Multiple fallbacks
  region = coalesce(var.override_region, var.aws_region, "us-east-1")
}

# coalescelist(list1, list2, ...) - returns first non-empty list
locals {
  subnet_ids = coalescelist(var.custom_subnet_ids,
data.aws_subnets.default.ids)
}
```

---

### `try` — Error-Safe Expression Evaluation

```
# try(expr1, expr2, ...) - evaluates exprs in order, returns first that
doesn't error

locals {
  # Safe attribute access on potentially null object
  db_port = try(var.database_config.port, 5432)

  # Safe map key access
  instance_type = try(local.instance_types[var.environment], "t3.micro")

  # Safe type conversion
  count_number = try(tonumber(var.count_string), 1)

  # Safe nested access
  vpc_id = try(data.aws_vpc.custom[0].id, aws_vpc.default.id)
}
```

```
# try() is especially useful with optional module inputs
variable "config" { type = any }

locals {
  timeout = try(var.config.timeout, 30)    # use config.timeout or 30
  retries = try(var.config.retries, 3)     # use config.retries or 3
}
```

---

### `can` — Test if Expression Succeeds

```
# can(expr) - returns true if expression evaluates without error, false
otherwise

variable "vpc_cidr" {
  type = string
  validation {
    condition     = can(cidrhost(var.vpc_cidr, 0))
    error_message = "Must be a valid CIDR block."
  }
}

locals {
  # Check if key exists in map
  has_prod_config = can(var.configs["prod"])

  # Check if value is valid number
  is_number = can(tonumber(var.string_value))
}
```

---

### Other Essential Functions

```
# String functions
lower("HELLO")           # "hello"
upper("hello")           # "HELLO"
trimspace("  hello  ")   # "hello"
replace("hello world", "world", "terraform")  # "hello terraform"
split(",", "a,b,c")     # ["a", "b", "c"]
join("-", ["a", "b"])    # "a-b"
format("%-10s %s", "hello", "world")  # "hello      world"
startswith("hello", "he")  # true
endswith("hello", "lo")    # true

# Number functions
min(3, 1, 2)             # 1
max(3, 1, 2)             # 3
abs(-5)                  # 5
ceil(1.2)                # 2
```

```
floor(1.9)               # 1

# Collection functions
length(["a", "b", "c"])  # 3
contains(["a", "b"], "a")  # true
keys({a=1, b=2})         # ["a", "b"]
values({a=1, b=2})       # [1, 2]
element(["a","b","c"], 1)  # "b"
slice(["a","b","c","d"], 1, 3)  # ["b", "c"]
concat(["a","b"], ["c","d"])    # ["a","b","c","d"]
distinct(["a","b","a"])         # ["a","b"]
reverse(["a","b","c"])          # ["c","b","a"]
sort(["c","a","b"])             # ["a","b","c"]
zipmap(["a","b"], [1,2])        # {a=1, b=2}

# Encoding functions
base64encode("hello")    # "aGVsbG8="
base64decode("aGVsbG8=") # "hello"
jsonencode({a = 1})      # "{\"a\":1}"
jsondecode("{\"a\":1}")  # {a = 1}

# Filesystem functions (see Topic 26)
file("path/to/file.txt")
templatefile("template.tpl", {var = "value"})

# IP/CIDR functions
cidrsubnet("10.0.0.0/16", 8, 0)   # "10.0.0.0/24"
cidrhost("10.0.0.0/24", 5)         # "10.0.0.5"
cidrnetmask("10.0.0.0/24")         # "255.255.255.0"

# Type conversion
tostring(42)             # "42"
tonumber("42")           # 42
tobool("true")           # true
tolist(toset(["a","b"])) # list from set
```

---

## 🔵 Short Interview Answer

> "The functions I use most in production: `merge()` for combining tag maps, `lookup()` for safe map access with a default fallback, `flatten()` for collapsing nested lists (especially from modules), `toset()` for converting lists to sets for `for_each`, `try()` for safe expression evaluation where the value might not exist, and `can()` in variable validations to test CIDR or other format validity. `coalesce()` for first-non-null patterns. These cover probably 90% of real-world Terraform function usage."

---

## 🔵 Gotchas & Edge Cases

- ⚠ **`lookup()` with no default errors** — `lookup(map, key)` without a default still errors if key is missing in many provider versions. Always provide a default.

- ⚠ `try()` **swallows all errors** — a mistake in the expression (typo in attribute name) silently falls back to the default. Use `try()` carefully and test both paths.
- `merge()` **shallow-merges** — nested maps are replaced, not deep-merged. `merge({a = {x=1}}, {a = {y=2}})` gives `{a = {y=2}}`, not `{a = {x=1, y=2}}`.
- `toset()` **loses order** — sets are unordered. Don't rely on the order of elements after `toset()`.
- `element()` **wraps around** — `element(["a","b","c"], 5)` returns `"c"` (index 5 % 3 = 2). Useful for AZ distribution but can be surprising.

---

# Topic 25: ⚠ Dynamic Blocks — When to Use, When to Avoid

---

## 🔵 What It Is (Simple Terms)

Dynamic blocks generate **repeated nested blocks** inside a resource based on a collection. They're the equivalent of a `for` loop for nested blocks — when you need 0 to N `ingress` rules, `ebs_block_device` entries, or `route` entries, dynamic blocks let you generate them from a variable.

---

## 🔵 Why It Exists — What Problem It Solves

```
# ❌ Without dynamic blocks — hardcoded repeated blocks
resource "aws_security_group" "web" {
  ingress {
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  ingress {
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  ingress {
    from_port   = 8080
    to_port     = 8080
    protocol    = "tcp"
    cidr_blocks = ["10.0.0.0/8"]
  }
  # What if we need 10 rules? Or 0 in some environments?
}
```

```
# ✅ With dynamic blocks — generated from variable
variable "ingress_rules" {
  type = list(object({
    port        = number
    protocol    = string
    cidr_blocks = list(string)
  }))
}

resource "aws_security_group" "web" {
  dynamic "ingress" {
    for_each = var.ingress_rules
    content {
      from_port   = ingress.value.port
      to_port     = ingress.value.port
      protocol    = ingress.value.protocol
      cidr_blocks = ingress.value.cidr_blocks
    }
  }
}
```

---

## 🔵 Dynamic Block Anatomy

```
dynamic "<BLOCK_TYPE>" {
  # ── for_each ──────────────────────────────────────────────────────
  for_each = <collection>         # list, map, or set to iterate over

  # ── iterator (optional) ───────────────────────────────────────────
  iterator = rule                 # name for the loop variable (default:
block type name)

  # ── labels (for blocks that need labels) ──────────────────────────
  # labels = [rule.key]           # for labeled nested blocks

  # ── content block ─────────────────────────────────────────────────
  content {
    # Access current item:
    # <iterator>.key    — map key or index
    # <iterator>.value  — map value or list item
    from_port   = rule.value.from_port
    to_port     = rule.value.to_port
    protocol    = rule.value.protocol
    cidr_blocks = rule.value.cidr_blocks
  }
}
```

---

## 🔵 Real World Examples

### Security Group Rules (Most Common)

```
variable "ingress_rules" {
  type = list(object({
    description = string
    from_port   = number
    to_port     = number
    protocol    = string
    cidr_blocks = list(string)
  }))
  default = [
    { description = "HTTP",  from_port = 80,  to_port = 80,  protocol =
"tcp", cidr_blocks = ["0.0.0.0/0"] },
    { description = "HTTPS", from_port = 443, to_port = 443, protocol =
"tcp", cidr_blocks = ["0.0.0.0/0"] },
  ]
}

resource "aws_security_group" "web" {
  name   = "web-sg"
  vpc_id = aws_vpc.main.id

  dynamic "ingress" {
    for_each = var.ingress_rules
    iterator = rule
    content {
      description = rule.value.description
      from_port   = rule.value.from_port
      to_port     = rule.value.to_port
      protocol    = rule.value.protocol
      cidr_blocks = rule.value.cidr_blocks
    }
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

### EBS Block Devices

```
variable "ebs_volumes" {
  type = list(object({
    device_name = string
    volume_size = number
    volume_type = string
  }))
```

```
  default = []
}

resource "aws_instance" "web" {
  ami           = data.aws_ami.ubuntu.id
  instance_type = "t3.medium"

  dynamic "ebs_block_device" {
    for_each = var.ebs_volumes
    content {
      device_name = ebs_block_device.value.device_name
      volume_size = ebs_block_device.value.volume_size
      volume_type = ebs_block_device.value.volume_type
      encrypted   = true
    }
  }
}
```

Optional Nested Block (Dynamic for 0 or 1)

```
# Dynamic block for an optional configuration
variable "enable_monitoring" {
  type    = bool
  default = false
}

resource "aws_instance" "web" {
  ami           = data.aws_ami.ubuntu.id
  instance_type = "t3.medium"

  # Create monitoring block only if enabled
  dynamic "metadata_options" {
    for_each = var.enable_imdsv2 ? [1] : []    # list with 1 item =
include block
    content {                                   # empty list = omit block
      http_endpoint               = "enabled"
      http_tokens                 = "required"
      http_put_response_hop_limit = 1
    }
  }
}

# Pattern: for_each = condition ? [1] : [] creates 0 or 1 blocks
# This is the idiomatic way to make a nested block optional
```

---

## 🔵 When to Use Dynamic Blocks

```
✅ USE dynamic blocks when:
  - The number of nested blocks varies (0 to N rules, volumes, routes)
  - The blocks come from a variable (user configures them)
  - You have identical block structure repeated multiple times
  - The block is genuinely optional based on a condition

❌ AVOID dynamic blocks when:
  - The blocks are static and known at write time
    (just write them out — clearer and simpler)
  - There are only 2-3 fixed configurations
    (dynamic adds complexity without benefit)
  - You're trying to work around a poorly structured resource
    (consider a different approach)
  - The content block logic is very complex
    (consider a separate resource with for_each instead)
```

---

## 🔵 Short Interview Answer

> "Dynamic blocks generate repeated nested configuration blocks from a collection, like a `for` loop for block content. The most common use case is security group ingress/egress rules — the number of rules varies by environment, so you define a variable with a list of rule objects and use `dynamic 'ingress'` with `for_each` to generate the blocks. The pattern `for_each = condition ? [1] : []` creates an optional block — zero items means the block is omitted. Avoid dynamic blocks when the nested blocks are static and known at write time — just write them explicitly for clarity."

---

## 🔵 Gotchas & Edge Cases

- ⚠ **Dynamic blocks can't be used for top-level blocks** — only for nested blocks inside resources. You can't use `dynamic "resource"` to create resource blocks.
- ⚠ **`iterator` defaults to the block name** — if block type is `ingress`, the loop variable is `ingress.value`. Use `iterator = rule` to rename for clarity.
- **Dynamic blocks don't support `count` or `for_each` meta-arguments** — you can't put meta-arguments inside `content {}`.
- **Nested dynamic blocks** — you can have a dynamic block inside another dynamic block's content. Gets complex quickly — consider refactoring.
- **Changing `for_each` collection** — adding, removing, or reordering items in the `for_each` collection regenerates all blocks. For security groups, this means all rules are re-evaluated on every plan.

---

## 🔵 Connections to Other Concepts

- → **Topic 23 (Expressions):** Dynamic blocks use `for_each` expressions
- → **Category 5 (Meta-arguments):** `for_each` is a meta-argument on resources; `for_each` inside dynamic is different
- → **Category 7 (Modules):** Dynamic blocks enable flexible module interfaces

---

# Topic 26: ➕ `templatefile()` and `file()` Functions

---

## 🔵 What It Is (Simple Terms)

`file()` reads a file's raw content as a string. `templatefile()` reads a file and renders it as a template — substituting variables and evaluating template directives. These are essential for generating user data scripts, cloud-init configs, policy documents, and configuration files.

---

## 🔵 `file()` — Read Raw File Content

```
# Syntax: file(path)
# Returns: file content as string

# Read a shell script
resource "aws_instance" "web" {
  ami           = data.aws_ami.ubuntu.id
  instance_type = "t3.medium"
  user_data     = file("${path.module}/scripts/init.sh")
}

# Read a public key
resource "aws_key_pair" "deployer" {
  key_name   = "deployer-key"
  public_key = file("${path.root}/keys/deployer.pub")
}

# Read a certificate
resource "aws_acm_certificate" "custom" {
  private_key       = file("${path.module}/certs/private.key")
  certificate_body  = file("${path.module}/certs/certificate.crt")
}
```

**Path functions — critical to understand:**

```
path.module   # Absolute path to the current MODULE directory
path.root     # Absolute path to the ROOT module directory
path.cwd      # Current working directory (where terraform is invoked)

# Always use path.module for files relative to the module
# This ensures the module works correctly when called from different
locations
user_data = file("${path.module}/scripts/init.sh")
```

---

## 🔵 `templatefile()` — Render Template Files

```
# Syntax: templatefile(path, variables)
# Returns: rendered template string

resource "aws_instance" "web" {
  ami           = data.aws_ami.ubuntu.id
  instance_type = "t3.medium"

  user_data = templatefile("${path.module}/templates/user_data.sh.tpl", {
    environment     = var.environment
    app_version     = var.app_version
    db_endpoint     = aws_db_instance.main.endpoint
    s3_bucket       = aws_s3_bucket.app.id
    log_level       = var.environment == "prod" ? "WARN" : "DEBUG"
  })
}
```

```
#!/bin/bash
# templates/user_data.sh.tpl

set -euo pipefail

# Environment configuration
export APP_ENV="${environment}"
export APP_VERSION="${app_version}"
export DB_HOST="${db_endpoint}"
export S3_BUCKET="${s3_bucket}"
export LOG_LEVEL="${log_level}"

# Install application
apt-get update -y
apt-get install -y nginx awscli

# Configure nginx
cat > /etc/nginx/sites-available/default << 'EOF'
server {
    listen 80;
    location /health { return 200 "ok"; }
    location / { proxy_pass http://localhost:8080; }
}
EOF

# Download and start application
aws s3 cp s3://${s3_bucket}/releases/${app_version}/app.tar.gz /opt/app/
tar -xzf /opt/app/app.tar.gz -C /opt/app/
systemctl start app
```

---

## 🔵 Template Directives (`%{...}`)

```
# templatefile supports directives for logic:

# Conditional
%{ if environment == "prod" ~}
export LOG_LEVEL="WARN"
%{ else ~}
export LOG_LEVEL="DEBUG"
%{ endif ~}

# For loop
%{ for server in backend_servers ~}
upstream_server ${server.ip}:${server.port};
%{ endfor ~}

# The ~ trims whitespace (newline) after the directive tag
```

```
# Terraform config
resource "aws_lb_target_group" "app" { ... }

resource "aws_instance" "nginx" {
  user_data = templatefile("${path.module}/templates/nginx.conf.tpl", {
    environment      = var.environment
    backend_servers  = [
      for inst in aws_instance.app : {
        ip   = inst.private_ip
        port = 8080
      }
    ]
  })
}
```

```
# templates/nginx.conf.tpl
upstream backend {
%{ for server in backend_servers ~}
    server ${server.ip}:${server.port};
%{ endfor ~}
}

server {
    listen 80;
    location / {
        proxy_pass http://backend;
    }
%{ if environment != "prod" ~}
    location /debug { return 200 "debug enabled"; }
```

```
%{ endif ~}
}
```

---

## 🔵 `templatefile()` vs Heredoc

```
# ❌ Heredoc — inline template in .tf file
user_data = <<-EOT
  #!/bin/bash
  export ENV="${var.environment}"
  export DB="${aws_db_instance.main.endpoint}"
  # Gets messy for complex scripts
EOT

# ✅ templatefile() — separate file, better for:
# - Long scripts (dozens of lines)
# - Complex logic with conditionals and loops
# - Syntax highlighting in editors
# - Reusable across multiple resources
# - Testable independently
user_data = templatefile("${path.module}/templates/init.sh.tpl", {
  environment = var.environment
  db_endpoint = aws_db_instance.main.endpoint
})
```

---

## 🔵 Common Use Cases

```
# Cloud-init config
resource "aws_instance" "web" {
  user_data = base64encode(templatefile("${path.module}/templates/cloud-
init.yaml.tpl", {
    hostname    = "web-${var.environment}"
    dns_servers = ["8.8.8.8", "8.8.4.4"]
    packages    = ["nginx", "awscli", "python3"]
  }))
}

# Kubernetes ConfigMap
resource "kubernetes_config_map" "app" {
  metadata { name = "app-config" }
  data = {
    "config.yaml" =
templatefile("${path.module}/templates/config.yaml.tpl", {
      db_host  = var.db_host
      redis_url = var.redis_url
    })
  }
}
```

```
# IAM policy document (when aws_iam_policy_document is too verbose)
resource "aws_iam_policy" "custom" {
  policy = templatefile("${path.module}/templates/policy.json.tpl", {
    account_id = data.aws_caller_identity.current.account_id
    bucket_arn = aws_s3_bucket.data.arn
  })
}

# ECS task definition
resource "aws_ecs_task_definition" "app" {
  container_definitions =
templatefile("${path.module}/templates/container-def.json.tpl", {
    image       = var.docker_image
    cpu         = var.container_cpu
    memory      = var.container_memory
    environment = var.environment
    log_group   = aws_cloudwatch_log_group.app.name
  })
}
```

---

## 🔵 Short Interview Answer

> "`file()` reads a file's raw content as a string — used for static files like public keys, certificates, or simple scripts. `templatefile()` renders a template file with variable substitution and directives (`%{if}`, `%{for}`) — the right tool for user data scripts, cloud-init configs, and any file that needs dynamic content. Always use `path.module` to reference files relative to the module rather than hardcoding paths. For complex scripts, `templatefile()` is preferred over heredoc strings in `.tf` files because the template lives in its own file with proper syntax highlighting and is independently testable."

---

## 🔵 Gotchas & Edge Cases

- ⚠ **`file()` at plan time** — `file()` is evaluated during plan, not apply. If the file doesn't exist, the plan fails. The file must be present in the module directory.
- ⚠ **Template variables must all be provided** — if your template uses `${db_endpoint}` but you don't pass `db_endpoint` in the variables map, `templatefile()` errors at plan time.
- **Dollar sign escaping** — in templatefile templates, literal `$` must be escaped as `$$` to prevent interpolation.
- **`base64encode()` for user_data** — many AWS resources require user_data as base64. Wrap: `base64encode(templatefile(...))`.
- **`path.module` vs `path.cwd`** — always use `path.module` for files bundled with a module. `path.cwd` is the Terraform invocation directory — unreliable when modules are called from different places.

---

# Topic 27: ➕ `formatdate()` and Date/Time Functions

## 🔵 What It Is (Simple Terms)

`formatdate()` formats a timestamp string into a human-readable or custom format. Terraform has a small set of time-related functions used primarily for resource naming, tagging with timestamps, and rotation schedules.

## 🔵 `timestamp()` and `formatdate()`

```
# timestamp() — returns current UTC time in RFC3339 format
# ⚠ Evaluated at plan time, changes every plan — use carefully

locals {
  # Current timestamp
  now = timestamp()
  # Returns: "2024-01-15T10:30:00Z"

  # Formatted for resource naming
  date_tag = formatdate("YYYY-MM-DD", timestamp())
  # Returns: "2024-01-15"

  # Formatted for log retention labels
  month_year = formatdate("MMMM YYYY", timestamp())
  # Returns: "January 2024"
}
```

## 🔵 `formatdate()` Format Specifiers

```
formatdate("YYYY",       "2024-01-15T10:30:00Z")  # "2024"
formatdate("YY",         "2024-01-15T10:30:00Z")  # "24"
formatdate("MM",         "2024-01-15T10:30:00Z")  # "01"
formatdate("MMM",        "2024-01-15T10:30:00Z")  # "Jan"
formatdate("MMMM",       "2024-01-15T10:30:00Z")  # "January"
formatdate("DD",         "2024-01-15T10:30:00Z")  # "15"
formatdate("hh",         "2024-01-15T10:30:00Z")  # "10"
formatdate("mm",         "2024-01-15T10:30:00Z")  # "30"
formatdate("ss",         "2024-01-15T10:30:00Z")  # "00"
formatdate("ZZZ",        "2024-01-15T10:30:00Z")  # "UTC"

# Combined formats
formatdate("YYYY-MM-DD",           "2024-01-15T10:30:00Z")  # "2024-01-15"
formatdate("DD MMM YYYY",          "2024-01-15T10:30:00Z")  # "15 Jan
2024"
formatdate("YYYY-MM-DD'T'hh:mm:ss", "2024-01-15T10:30:00Z") # "2024-01-
15T10:30:00"
```

## 🔵 `timeadd()` — Time Arithmetic

```
# timeadd(timestamp, duration) — add duration to timestamp
# Duration format: "Xh", "Xm", "Xs", "Xhour", etc.

locals {
  now         = timestamp()
  one_hour    = timeadd(timestamp(), "1h")
  one_day     = timeadd(timestamp(), "24h")
  one_week    = timeadd(timestamp(), "168h")
  minus_1_day = timeadd(timestamp(), "-24h")
}
```

---

## 🔵 `timecmp()` — Compare Timestamps

```
# timecmp(ts1, ts2) — returns -1 (ts1 < ts2), 0 (equal), 1 (ts1 > ts2)

locals {
  expiry_date = "2024-12-31T00:00:00Z"
  is_expired  = timecmp(timestamp(), local.expiry_date) > 0
}
```

---

## 🔵 Real World Usage Patterns

```
# Tagging resources with creation date
resource "aws_s3_bucket" "data" {
  bucket = "myapp-data-${var.environment}"

  tags = {
    Name        = "myapp-data-${var.environment}"
    CreatedAt   = formatdate("YYYY-MM-DD", timestamp())
    Environment = var.environment
  }
}

# ⚠ WARNING: timestamp() changes on every plan
# The above will show a "change" every plan because CreatedAt updates
# SOLUTION: use ignore_changes
resource "aws_s3_bucket" "data" {
  tags = {
    CreatedAt = formatdate("YYYY-MM-DD", timestamp())
  }
  lifecycle {
    ignore_changes = [tags["CreatedAt"]]    # Don't update CreatedAt after
creation
```

```
  }
}

# TLS certificate expiry check
resource "tls_private_key" "cert" {
  algorithm = "RSA"
  rsa_bits  = 4096
}

# Rotation schedule naming (for backup policies)
resource "aws_backup_plan" "daily" {
  name = "daily-backup-${formatdate("YYYY-MM", timestamp())}"
  rule {
    rule_name         = "daily"
    target_vault_name = aws_backup_vault.main.name
    schedule          = "cron(0 2 * * ? *)"
    retention_days    = 30
  }
}
```

---

## 🔵 Short Interview Answer

> "`formatdate()` converts a timestamp string to a custom format using format specifiers like `YYYY-MM-DD`. It's typically used with `timestamp()` for tagging resources with creation dates or building time-based names. The key gotcha is that `timestamp()` is evaluated at every plan, so a tag with `timestamp()` will appear to change on every plan — use `lifecycle { ignore_changes = [tags[\"CreatedAt\"]] }` to prevent this. `timeadd()` does time arithmetic and `timecmp()` compares timestamps."

---

## 🔵 Gotchas & Edge Cases

- ⚠ **`timestamp()` changes every plan** — any resource attribute using `timestamp()` will show as changed on every `terraform plan`. Always pair with `ignore_changes` or use it only for resource creation.
- **`timestamp()` is UTC only** — there's no timezone conversion. If your organization tags with local time, you'd need to calculate the offset manually with `timeadd()`.
- **Format string is case-sensitive** — `YYYY` is year, `yyyy` is also year in some contexts. Stick to the documented uppercase format specifiers.

---

# Topic 28: ➕ Tuple Type — How It Differs from List, When Terraform Returns Tuples

---

## 🔵 What It Is (Simple Terms)

A **tuple** is a fixed-length sequence where each element can be a **different type**. It's like a list, but with a strict schema — position 0 is always a string, position 1 is always a number, etc.

```
# List: variable length, all same type
list(string)   →  ["a", "b", "c"]      # any length, all strings

# Tuple: fixed length, mixed types
tuple([string, number, bool])  →  ["web", 8080, true]  # exactly 3, mixed
types
```

---

## 🔵 Why Tuples Matter in Practice

You rarely **declare** tuple variables intentionally — but Terraform **returns** tuples from certain expressions, and understanding this prevents type errors.

---

## 🔵 When Terraform Returns Tuples

### `for` Expression with Mixed Types

```
# This returns a TUPLE, not a list
locals {
  mixed = ["web", 8080, true]          # tuple([string, number, bool])
  # Different from:
  strings = ["web", "api", "worker"]   # list(string)
}
```

### Splat on `for_each` Resources

```
# for_each returns a map — splat may return tuple
resource "aws_instance" "web" {
  for_each = var.instances
}

# This might return a tuple in some contexts:
output "ids" {
  value = values(aws_instance.web)[*].id
}
```

### `concat()` with Mixed Types

```
# concat() returns a tuple when inputs have different element types
locals {
  mixed = concat(["a", "b"], [1, 2])   # returns tuple([string, string,
```

```
number, number])
}
```

---

## 🔵 Tuple vs List — When It Matters

```
# The problem: you expect list(string) but get tuple
variable "subnet_ids" {
  type = list(string)        # expects a list
}

locals {
  # This returns a TUPLE if any element differs in type:
  ids = [aws_subnet.private.id, aws_subnet.public.id]  # Actually
list(string) - fine
}

# Converting tuple to list when needed:
locals {
  ids_as_list = tolist([aws_subnet.a.id, aws_subnet.b.id])
}

# For variables expecting list(string), Terraform auto-converts
# compatible tuples — usually not an issue for same-type elements
```

---

## 🔵 Practical: `object` vs `tuple` for Return Types

```
# When a provider or function returns a complex type,
# Terraform may use tuples internally

# cidrsubnets() returns a tuple of strings
locals {
  subnets = cidrsubnets("10.0.0.0/16", 8, 8, 8)
  # Returns: tuple(["10.0.0.0/24", "10.0.1.0/24", "10.0.2.0/24"])
  # Indexing works the same as lists: local.subnets[0] = "10.0.0.0/24"
}

# for expression returns tuple when types are mixed
locals {
  # Same type elements — effectively a list
  ports = [for svc in var.services : svc.port]     # all numbers —
list(number)
  # Mixed type elements — tuple
  configs = [for svc in var.services : [svc.name, svc.port]]  # nested
tuples
}
```

## 🔵 `tolist()` and `totuple()` — Type Conversions

```
# Convert tuple to list (when you need list type for a variable)
locals {
  tuple_value = ["a", "b", "c"]         # Terraform may type this as tuple
  list_value  = tolist(["a", "b", "c"]) # Explicit list(string)
}

# Error scenario and fix:
variable "names" { type = list(string) }

# This might error if Terraform infers tuple:
# names = [resource1.name, resource2.name]  # could be tuple

# Fix: explicit tolist()
# names = tolist([resource1.name, resource2.name])
```

---

## 🔵 Short Interview Answer

> "A tuple is a fixed-length sequence where elements can be different types — like `[\"web\", 8080, true]` — compared to a list where all elements must be the same type. In practice, you rarely declare tuples explicitly — but Terraform returns tuples from certain expressions like `for` loops that produce mixed types, `cidrsubnets()`, and `concat()` with different input types. The common issue is passing a tuple where `list(string)` is expected — fix with `tolist()`. Understanding tuples helps debug 'expected list of string, got tuple' type errors."

---

## 🔵 Gotchas & Edge Cases

- ⚠ **"Expected list of string, got tuple"** — this error appears when Terraform infers a tuple where a `list(string)` is expected. Fix: wrap with `tolist()`.
- ⚠ **Tuple indexing works like lists** — `tuple_val[0]` works fine. The difference is in type constraints and what operations accept tuples vs lists.
- **`for` expressions with consistent types return list** — if all elements in `[for x in list : x.attribute]` are the same type, Terraform often infers `list(type)` not `tuple`. Mixed types are what triggers tuple.
- **Tuples can't be iterated in `for_each`** — `for_each` requires `set` or `map`. Convert tuples to sets with `toset()` or maps with a `{for}` expression.

---

---

# 📊 Category 4 Summary — Quick Reference Card

| Topic | One-Line Summary | Interview Weight |
|---|---|---|
| 18. Input Variables | Types, validation, sensitive — required vs optional | ⭐⭐⭐⭐ |

| Topic | One-Line Summary | Interview Weight |
|---|---|---|
| 19. Outputs | Expose values, cross-module/stack communication | ⭐⭐⭐⭐ |
| 20. Locals | Computed reusable expressions — reduce repetition | ⭐⭐⭐⭐ |
| 21. Precedence ⚠ | TF_VAR > -var > auto.tfvars > tfvars > default | ⭐⭐⭐⭐⭐ |
| 22. tfvars | Auto-load vs explicit, never secrets in files | ⭐⭐⭐⭐ |
| 23. Expressions | Ternary, for, splat — for_each splat gotcha | ⭐⭐⭐⭐ |
| 24. Functions | merge, lookup, flatten, toset, try, coalesce | ⭐⭐⭐⭐⭐ |
| 25. Dynamic Blocks ⚠ | Repeat nested blocks from collection | ⭐⭐⭐⭐ |
| 26. templatefile/file | Template rendering for user_data, configs | ⭐⭐⭐ |
| 27. formatdate | Timestamp formatting — ignore_changes gotcha | ⭐⭐ |
| 28. Tuple Type | Fixed-length mixed-type sequence, tolist() fix | ⭐⭐⭐ |

---

## 🔑 Category 4 — Critical Rules to Remember

```
Variables:
  No default = required            |  default = null = optional (nullable)
  sensitive = true = redacted CLI  |  but still plaintext in state

Precedence (lowest → highest):
  default → tfvars → auto.tfvars → -var-file → -var → TF_VAR_

Locals:
  Can ref other locals ✅          |  Cannot be set from outside ❌
  Can ref resources ✅             |  Circular refs error ❌

Expressions:
  Splat [*] for count ✅           |  Splat [*] for for_each ❌  (use
values()[*])
  for [...] = list                 |  for {...} = map

Dynamic blocks:
  for_each = condition ? [1] : [] →  optional block (0 or 1 instances)

templatefile():
  Always use path.module           |  All template vars must be passed
  base64encode() for user_data     |  Escape literal $ as $$

timestamp():
  Changes every plan               |  Use ignore_changes to prevent drift
```

---

# 🎯 Category 4 — Top 5 Interview Questions to Master

---

1. **"What is the variable precedence order in Terraform?"** — full chain, TF_VAR is highest not lowest
2. **"What does `sensitive = true` on a variable actually do?"** — redacts CLI output only, still in state
3. **"When would you use a local vs a variable?"** — local = internal computed, variable = external input
4. **"What are dynamic blocks and when should you avoid them?"** — repeating nested blocks, avoid for static known configs
5. **"How do you safely access a map key that might not exist?"** — `lookup(map, key, default)` or `try(map[key], default)`

---

> **Next:** Category 5 — Meta-Arguments & Lifecycle (Topics 29–34) Type `Category 5` to continue, `quiz me` to be tested on Category 4, or `deeper` on any specific topic.